



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

P.F.G: Ingeniería en Informática

Unidad Curricular: Lenguaje y Compiladores

Tema 2:

Los lenguajes de programación

Profesor :

Félix Márquez

Alumnos:

Shirley Cedeño V-30.413.183 S2

Nelson Bueno V-27.407.263 S2

Angel Rodriguez V-30.437.205 S1

Rayc Yanez V-28.215.618 S2

Ciudad Guayana, 30 mayo del 2026

Resumen Académico

El presente informe de investigación analiza la infraestructura lingüística y computacional de las herramientas de desarrollo modernas, trascendiendo la perspectiva del programador aplicativo hacia el diseño arquitectónico de compiladores. El propósito central del estudio es evaluar cómo las decisiones de diseño léxico, morfológico y paradigmático impactan directamente en la eficiencia y ejecución del software. Para contrastar estas filosofías, se analizaron cuatro lenguajes con aproximaciones divergentes: Zig y Rust (compilación nativa directa), JavaScript (compilación *Just-In-Time* sobre el motor V8) y Python (interpretado puro mediante máquina virtual). La medición de rendimiento se abordó resolviendo un problema de procesamiento matemático intensivo basado en la Conjetura de Collatz bajo un límite estricto de 10,000,000 de iteraciones. Los resultados empíricos demostraron una asimetría arquitectónica significativa: mientras las soluciones nativas (Zig y Rust) culminaron la prueba en ~1.4 segundos con perfiles mínimos de memoria, los entornos gestionados presentaron una paradoja de recursos. JavaScript sacrificó RAM física (39.05 MB) a cambio de velocidad (~16.8 s), mientras que Python minimizó el consumo interno pero sufrió una degradación temporal masiva (~26 minutos). Finalmente, se presenta el diseño abstracto del Lenguaje L, un Lenguaje de Dominio Específico (DSL) innovador orientado al sistema de gestión de microrredes eléctricas ECO-GRID. Este DSL consolida el aprendizaje teórico al proponer una gramática resiliente y legible en español, abstrayendo el control físico de actuadores y prevención de fugas térmicas para operadores en entornos industriales críticos.

Introducción

En el ámbito de las ciencias de la computación y la ingeniería de software, un lenguaje de programación trasciende su función básica como un simple puente de comunicación entre el ser humano y la máquina. Históricamente, en las etapas iniciales de la formación informática, los estudiantes interactúan con estos lenguajes exclusivamente desde la perspectiva del usuario final, utilizándolos como herramientas estáticas para traducir lógica algorítmica y resolver problemas estructurados. Sin embargo, al adentrarse en la teoría de autómatas y el diseño de compiladores, surge un planteamiento conceptual mucho más profundo: el lenguaje debe ser comprendido como un producto de software en sí mismo, minuciosamente diseñado, sujeto a restricciones formales rigurosas y construido con una arquitectura orientada a fines específicos.

Para desglosar y comprender el comportamiento de estas herramientas, es imperativo delimitar sus niveles de infraestructura lingüística. En la base se encuentra el nivel morfológico y léxico, responsable de la tokenización determinista del código fuente. Esta fase define cómo el analizador agrupa caracteres aislados en unidades con significado atómico (palabras reservadas, reglas de identificadores, literales) y cómo trata los elementos irrelevantes o la indentación. Sobre este cimiento opera el nivel sintáctico, el cual establece el orden gramatical estricto que rige la jerarquía de las estructuras de control. Estas capas no representan simples estilos de escritura, sino que dictaminan cómo el compilador construirá el Árbol de Sintaxis Abstracta (AST) para la posterior traducción semántica y generación de código máquina.

En la actualidad, analizar la eficiencia estructural del software posee una relevancia crítica. El desarrollo de sistemas modernos exige la convergencia multiparadigma, donde la

eficiencia no solo se mide por la complejidad algorítmica $O(n)$, sino por el costo arquitectónico real que el entorno de ejecución impone sobre el hardware.

El presente informe aborda esta transición teórica y práctica de manera integral. A través del análisis descriptivo de paradigmas, la ejecución de pruebas empíricas de estrés matemático (benchmarking) y la propuesta arquitectónica de un Lenguaje de Dominio Específico (DSL) para la gestión de microrredes eléctricas, se demostrará cómo las decisiones de diseño desde el nivel léxico impactan directamente en la expresividad, mantenibilidad y rendimiento del software a gran escala.

Actividad I: Análisis de Paradigmática

La Convergencia Multiparadigma en la Industria Actual

En el ecosistema contemporáneo de las ciencias de la computación y la ingeniería de software, la concepción de lenguajes de programación rígidamente adscritos a un único paradigma teórico ha quedado en la obsolescencia. Históricamente, el diseño de lenguajes se estructuraba bajo fronteras conceptuales estrictas, clasificando las herramientas en entornos puramente funcionales, imperativos o lógicos. Sin embargo, la creciente complejidad de los sistemas informáticos modernos, la necesidad de procesamiento escalable y los requerimientos de mantenibilidad a largo plazo han forzado a la industria hacia una arquitectura de convergencia multiparadigma.

Esta evolución arquitectónica responde a una necesidad puramente pragmática. Los ingenieros de software se enfrentan diariamente a problemas complejos que requieren enfoques mixtos. Por ejemplo, en el desarrollo de arquitecturas empresariales, es imperativo establecer una base estructural utilizando los principios de la Orientación a Objetos para modelar la lógica del negocio. No obstante, al momento de realizar transformaciones de datos complejas o consultas en memoria, el lenguaje adopta características del paradigma funcional, permitiendo un manejo de colecciones mucho más declarativo, seguro y con menos líneas de código propensas a errores.

Desde la perspectiva del diseño de compiladores, esta hibridación representa un desafío monumental. Un compilador moderno no solo debe traducir instrucciones secuenciales a código máquina o código intermedio, sino que debe poseer analizadores semánticos capaces de inferir tipos, manejar cierres funcionales en la memoria dinámica y resolver jerarquías de herencia complejas simultáneamente. La convergencia multiparadigma

demuestra que las decisiones de diseño léxico, morfológico y sintáctico no son meros estilos de escritura, sino infraestructuras que impactan de manera directa y medible en la eficiencia de ejecución, la asignación de memoria y la capacidad del compilador para realizar optimizaciones avanzadas de código.

Análisis Descriptivo y Arquitectónico de Paradigmas

Fundamentales

Para comprender la mecánica interna de las herramientas de desarrollo, es fundamental diseccionar los paradigmas de programación no solo desde su sintaxis, sino desde su impacto en el modelo de ejecución, la gestión de memoria y la teoría de autómatas que los sustenta.

1. Paradigma Imperativo/Estructural

El paradigma imperativo es la manifestación de alto nivel más directa de la arquitectura computacional de Von Neumann. Su ontología se basa en la gestión explícita del estado del sistema y la secuenciación estricta de instrucciones. En este modelo, el programa es conceptualizado como una serie de sentencias que cambian el estado de la memoria a través de asignaciones directas.

Desde el punto de vista de la teoría de compiladores, las variables en un lenguaje imperativo son abstracciones matemáticas de celdas de memoria física. El rasgo crítico de este paradigma es la mutabilidad de la memoria, lo cual genera de forma inherente efectos secundarios. Cuando una instrucción muta una variable global o un estado fuera de su ámbito local, el estado del sistema cambia impredeciblemente para otras rutinas concurrentes.

Para el diseñador de compiladores, el código imperativo exige un riguroso análisis de flujo de control y análisis de flujo de datos. El compilador debe rastrear minuciosamente la vida útil de cada variable para optimizar la asignación de registros en la CPU. La presencia constante de efectos secundarios limita drásticamente la capacidad del compilador para reordenar instrucciones o paralelizar bloques de código de manera automática, ya que el orden secuencial estricto debe preservarse para garantizar la correctitud del programa final.

2. Paradigma Orientado a Objetos (POO)

Nacido de la necesidad de dominar la complejidad exponencial en sistemas de gran escala, el Paradigma Orientado a Objetos propone una abstracción fundamentada en la cohesión de datos y comportamiento en entidades únicas. En la ingeniería de software aplicada, la POO permite mapear entidades del mundo real directamente a las estructuras del código.

Sus pilares fundamentales definen cómo el compilador debe organizar la memoria y resolver llamadas a funciones:

- **Encapsulamiento:** Restringe el acceso no autorizado al estado interno y obliga al compilador a generar reglas estrictas en la tabla de símbolos para validar ámbitos de visibilidad durante el análisis semántico.
- **Polimorfismo y Herencia:** Permite que entidades de diferentes tipos respondan a una misma interfaz. El compilador resuelve esto mediante la generación de Tablas de Métodos Virtuales, donde la dirección de memoria de la función a ejecutar se decide en tiempo de ejecución, añadiendo una capa de indirección que afecta el rendimiento.

El debate contemporáneo más relevante dentro de la POO es herencia contra composición. Mientras la herencia clásica genera jerarquías de tipos rígidas y fuertemente

acopladas, la composición favorece la inyección de dependencias y el ensamblaje de comportamientos pequeños y modulares, facilitando enormemente el mantenimiento del software.

3. Paradigma Funcional

Con profundas raíces matemáticas en el Cálculo Lambda, el paradigma funcional aborda la computación como la evaluación de expresiones y la aplicación de funciones matemáticas puras, buscando la eliminación programática de efectos colaterales.

Para lograrlo, impone la inmutabilidad estricta de los datos. Una variable, una vez instanciada, no puede cambiar su valor. Esto garantiza una propiedad invaluable conocida como transparencia referencial: una función llamada con los mismos argumentos siempre producirá el mismo resultado, independientemente del estado global del programa.

Al garantizar que no hay efectos secundarios, el compilador puede emplear agresivamente técnicas como la memorización, la eliminación de código muerto, la reordenación de instrucciones y la paralelización automática de rutinas en múltiples núcleos del procesador sin requerir semáforos o bloqueos. Asimismo, se apoya en conceptos como funciones como ciudadanos de primer orden y la evaluación perezosa, donde el compilador posterga el cálculo de una expresión hasta el instante exacto en que su resultado es estrictamente necesario, optimizando el uso de la memoria RAM.

4. Paradigma Lógico / Declarativo

El paradigma lógico representa la abstracción total del flujo de control por parte del programador. El desarrollador se limita a describir la naturaleza del problema, las reglas lógicas y el estado deseado de los datos, delegando al entorno de ejecución la tarea de encontrar la secuencia algorítmica para resolverlo.

El ejemplo más cotidiano y poderoso de este paradigma se evidencia en la administración de bases de datos relacionales y el lenguaje SQL. Al desarrollar procedimientos almacenados complejos o ejecutar migraciones masivas de datos, el ingeniero declara el conjunto de datos requerido, mientras que el motor subyacente genera un plan de ejecución optimizado basado en estadísticas de uso y teoría de conjuntos para retornar la información requerida.

A nivel de lenguajes puros, este paradigma funciona mediante la programación basada en relaciones, la aplicación del algoritmo de unificación matemática y la resolución de cláusulas de Horn. El intérprete construye un árbol de búsqueda y utiliza técnicas de retroceso heurístico para probar combinaciones lógicas hasta encontrar valores que satisfagan todas las reglas declaradas.

5. Paradigma Concurrente / Basado en Actores (Emergente)

Ante el estancamiento de la velocidad bruta de un solo núcleo de procesamiento, la industria ha escalado hacia arquitecturas de múltiples núcleos. Sin embargo, aplicar paralelismo en lenguajes imperativos tradicionales mediante hilos y memoria compartida introduce vulnerabilidades críticas como condiciones de carrera e interbloqueos.

El Paradigma Basado en Actores emerge como una solución arquitectónica para la mitigación de condiciones de carrera a nivel de diseño lingüístico, eliminando de forma absoluta la memoria compartida a través de modelos de paso de mensajes asíncronos.

El bloque de construcción es el "Actor", una entidad concurrente primitiva que posee su propio estado aislado y un buzón de entrada. Los actores no pueden modificar ni leer el estado de otros actores de forma directa; la única manera de interactuar es enviando un mensaje inmutable. El compilador y la máquina virtual se encargarán de encolar los mensajes

y garantizar que cada actor procese un solo mensaje a la vez de forma secuencial, garantizando un aislamiento estricto de estado y permitiendo construir sistemas altamente tolerantes a fallos.

Implicaciones Lingüísticas y Compilación

El análisis de estos paradigmas confirma que el lenguaje de programación es un producto arquitectónico minuciosamente diseñado. El Árbol de Sintaxis Abstracta (AST) que genera el analizador sintáctico difiere enormemente dependiendo del paradigma evaluado:

- En un entorno imperativo, el AST es denso en nodos de asignación y bucles, obligando al generador de código a ser extremadamente cuidadoso con la superposición de registros de CPU.
- En un entorno puramente funcional, el AST es un grafo de expresiones matemáticas anidadas, donde el analizador léxico a menudo ignora delimitadores explícitos, y el analizador semántico debe soportar inferencia de tipos paramétrica rigurosa.

Matriz Comparativa de Paradigmas Fundamentales

Paradigma	Fundamentos y Arquitectura Lógica	Tratamiento del Estado y Memoria	Gestión de Control y Efectos
Imperativo / Estructural	Abstracción de la Máquina de Von Neumann. Secuenciación de instrucciones explícitas.	Mutabilidad Inherente: La memoria se reasigna continuamente. Requiere monitoreo complejo del compilador.	Flujo por bucles y saltos. Genera efectos secundarios constantemente.
Orientado a Objetos (POO)	Modelado mediante la cohesión de estado y comportamiento.	Estado Controlado: Mutaciones ocurren únicamente a través	Envío de mensajes síncronos entre objetos. Resoluciones

	Encapsulamiento, polimorfismo.	de interfaces definidas (métodos).	mediante tablas de métodos virtuales.
Funcional	Cálculo lambda. Funciones de orden superior y evaluación perezosa para optimización de recursos.	Inmutabilidad Estricta: La memoria no se sobrescribe. Se generan nuevas estructuras en el heap.	Transparencia referencial. Eliminación de efectos secundarios, permitiendo paralelización automática.
Lógico / Declarativo	Aserciones de hechos y reglas lógicas. Resolución de cláusulas de Horn y unificación.	Estado Abstraído: El programador no manipula la memoria; el motor gestiona la memoria de trabajo temporal.	Inversión de control. El flujo algorítmico es delegado al motor de inferencia interno.
Concurrente / Actores	Diseñado para arquitecturas de múltiples núcleos. Sistemas de enrutamiento de mensajes.	Aislamiento Estricto: Ausencia total de memoria compartida entre procesos concurrentes.	Paso de mensajes asíncronos. Mitigación de condiciones de carrera e interbloqueos.

Actividad II: Estudio y Benchmarking

Análisis Morfológico (Léxico) y Sintáctico

Nota técnica preliminar: La tokenización analizada en esta sección corresponde a la teoría de lenguajes formales (donde un analizador léxico o scanner agrupa secuencias de caracteres en tokens o unidades atómicas de significado para el compilador). Este proceso es estrictamente determinista y gramatical, difiriendo por completo de la tokenización probabilística o por sub-palabras empleada en el Procesamiento de Lenguaje Natural (PLN) para modelos de inteligencia artificial.

1. Zig

Análisis Morfológico: Zig posee un analizador léxico estricto y minimalista. Sus palabras reservadas (como `fn`, `while`, `if`, `else`, `var`, `const`) están limitadas para evitar ambigüedades. Las reglas de identificadores exigen comenzar con una letra o guion bajo, seguido de caracteres alfanuméricos (`[a-zA-Z_][a-zA-Z0-9_]*`). Los literales numéricos soportan tipado fuerte implícito y los literales de cadena utilizan comillas dobles. Los elementos irrelevantes (espacios en blanco, tabulaciones y saltos de línea) son ignorados por el compilador, ya que Zig depende de delimitadores explícitos: utiliza llaves `{ }` para definir bloques de alcance léxico (*scope*) y el punto y coma `;` para la terminación de sentencias.

Análisis Sintáctico: A continuación, la notación formal (EBNF) de las estructuras de control empleadas:

```

<Subprograma> ::= "fn" <Identificador> "(" [ <ListaParametros> ] ")"
<TipoRetorno> "{" <BloqueInstrucciones> "}"
<BucleWhile> ::= "while" "(" <ExpresionBooleana> ")" "{" <BloqueInstrucciones>
"}"
<Condicional> ::= "if" "(" <ExpresionBooleana> ")" "{" <BloqueInstrucciones> "}"
[ "else" "{" <BloqueInstrucciones> "}" ]

```

2. Python

Análisis Morfológico: El escáner léxico de CPython tiene una característica arquitectónica única: el espacio en blanco es sintácticamente significativo. A diferencia de los otros lenguajes evaluados, Python no ignora la indentación; el analizador léxico inyecta *tokens* invisibles llamados **INDENT** y **DEDENT** para marcar el inicio y fin de los bloques, eliminando la necesidad de llaves. Sus palabras reservadas incluyen **def**, **while**, **if**, **else**. Los literales numéricos son tratados como objetos de precisión arbitraria por defecto. El delimitador explícito principal es los dos puntos **:** que anuncia la apertura de un nuevo bloque indentado.

Análisis Sintáctico: Representación formal de sus estructuras, destacando la obligatoriedad de los tokens de indentación:

```

<Subprograma> ::= "def" <Identificador> "(" [ <ListaParametros> ] "):"
<NUEVA_LINEA> <INDENT> <BloqueInstrucciones> <DEDENT>
<BucleWhile> ::= "while" <ExpresionBooleana> ":" <NUEVA_LINEA> <INDENT>
<BloqueInstrucciones> <DEDENT>
<Condicional> ::= "if" <ExpresionBooleana> ":" <NUEVA_LINEA> <INDENT>
<BloqueInstrucciones> <DEDENT>
[ "else" ":" <NUEVA_LINEA> <INDENT> <BloqueInstrucciones>
<DEDENT> ]

```

3. Rust

Análisis Morfológico: El análisis léxico de Rust está diseñado para garantizar la seguridad de memoria desde la fase de compilación. Las palabras reservadas (**fn**, **while**, **if**, **let**, **mut**) definen rígidamente la inmutabilidad por defecto. Las reglas de identificadores siguen el estándar alfanumérico ASCII/Unicode. Los literales son estrictos e infieren el tipo si no se sufijan (ej. **i32**, **u64**). El tratamiento de elementos irrelevantes es tradicional: los espacios y saltos de línea se ignoran. Utiliza delimitadores explícitos (**{ }** para bloques y **;** para sentencias). Una particularidad léxica es que el *scanner* no requiere (ni recomienda) paréntesis aislantes alrededor de las expresiones booleanas en sus estructuras de control.

Análisis Sintáctico: Notación formal evidenciando la ausencia de paréntesis en las condiciones de evaluación:

```
<Subprograma> ::= "fn" <Identificador> "(" [ <ListaParametros> ] ")" [ "->"  
<TipoRetorno> ] "{" <BloqueInstrucciones> }"  
<BucleWhile> ::= "while" <ExpresionBooleana> "{" <BloqueInstrucciones> }"  
<Condicional> ::= "if" <ExpresionBooleana> "{" <BloqueInstrucciones> }"  
                [ "else" ( <Condicional> | "{" <BloqueInstrucciones> }" ) ]
```

4. JavaScript

Análisis Morfológico: El escáner del motor V8 (ECMAScript) es altamente permisivo. Sus palabras reservadas empleadas incluyen **function**, **while**, **if**, **let**, **const**. Las reglas de identificadores permiten caracteres estándar y símbolos

especiales como $\$$ o $_$. Los literales numéricos, históricamente, se tokenizan siempre como flotantes de doble precisión, a menos que se declare explícitamente el sufijo n para BigInt. Emplea delimitadores explícitos ($\{ \}$ para bloques). Una característica crítica de su analizador léxico es el mecanismo ASI (*Automatic Semicolon Insertion*), donde el compilador infiere e inyecta virtualmente el delimitador $;$ al final de una línea si el programador lo omite, aunque el espacio en blanco en general sea ignorado.

Análisis Sintáctico: Notación formal de la jerarquía de estructuras de control en su paradigma base:

```
<Subprograma> ::= "function" <Identificador> "(" [ <ListaParametros> ] ")" "{"  
<BloqueInstrucciones> "}"  
<BucleWhile> ::= "while" "(" <ExpresionBooleana> ")" "{" <BloqueInstrucciones> "}"  
<Condicional> ::= "if" "(" <ExpresionBooleana> ")" <InstruccionOBloque>  
[ "else" <InstruccionOBloque> ]
```

Benchmarking en Procesamiento Intensivo

Algoritmo Seleccionado: Conjetura de Collatz aplicando un límite estricto de estrés de 10,000,000 de iteraciones consecutivas, evaluando los enteros en el rango de 1 a 10^7 . Cada lenguaje calcula la longitud de la secuencia matemática para cada elemento y actualiza el registro de la longitud máxima encontrada de forma secuencial.

- **Resultado de control (Secuencia máxima validada):** 685 pasos (idéntico en los cuatro entornos, lo que valida la integridad matemática del escenario de pruebas).

Entorno de Pruebas (Hardware/Software):

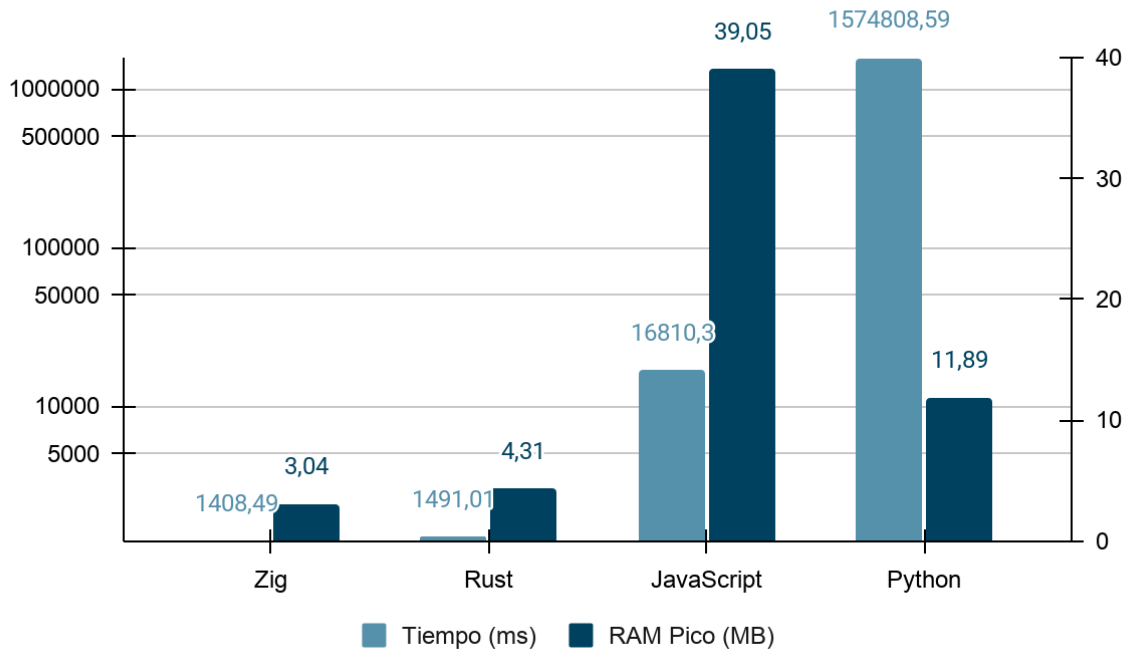
- **Arquitectura del Sistema:** Computador portátil HP ProBook 450 G6.

- **CPU:** Intel Core i5-8265U a 1.60GHz (hasta 3.90 GHz con Turbo Boost, 4 núcleos / 8 hilos).
- **Memoria RAM:** 16GB DDR4 a 2400 MHz.
- **Sistema Operativo:** Windows 11 Pro (64 bits).
- **Metodología de Medición:**
 - *Métricas Internas:* Capturadas mediante APIs nativas de introspección de cada lenguaje (`time.perf_counter()` y `tracemalloc` en Python; `performance.now()` y `process.memoryUsage().heapUsed` en JavaScript/Node.js; `std.time.Instant` en Rust).
 - *Métricas Externas (OS):* Auditadas mediante un script de supervisión en PowerShell ejecutado de manera concurrente, capturando la propiedad de hardware `WorkingSet64` (RAM física asignada al proceso) con un intervalo de muestreo agresivo de 10 milisegundos y un cronómetro de precisión de la clase `System.Diagnostics.Stopwatch`.

Tabla de Tiempos y Memoria

Lenguaje de Compilación	Paradigma Dominante	Mecanismo de Ejecución y Compilación	Tiempo de Ejecución Real OS (ms)	Consumo de Memoria Interno (nHeap)	Consumo de Memoria Pico Real OS
Zig	Imperativo / Estructurado	Compilación Nativa (LLVM)	1,408.49	0.00 MB	3.04 MB
Rust	Multiparadigma	Compilación Nativa (LLVM)	1,491.01	0.00 MB	4.31 MB
JavaScript	Multiparadigma	Compilación JIT (Node.js)	16,810.30	4.70 MB	39.05 MB
Python	Multiparadigma	Interpretado Puro (CPython)	1,574,808.59	0.00 MB	11.89 MB

Grafico comparativo de Tiempos y Memoria



Análisis Técnico y Discusión de Resultados

El estudio empírico revela un comportamiento asimétrico altamente dependiente de la arquitectura interna de los lenguajes evaluados, evidenciando cómo la gestión de memoria y el mecanismo de ejecución dictan el rendimiento final.

1. El Impacto de la Compilación Nativa (Zig y Rust)

Zig y Rust exhiben una eficiencia superior con tiempos de ejecución inferiores a los 1.5 segundos. Ambos lenguajes emplean LLVM como infraestructura de compilación. Al procesar un bucle matemático puro e iterativo, el optimizador de LLVM traduce las estructuras de control directamente a instrucciones vectorizadas u optimizadas a nivel de registro de CPU en lenguaje ensamblador x86_64.

Respecto a la memoria, ambos registran 0.00 MB de uso en el Heap interno debido a que la complejidad espacial del algoritmo es constante $O(1)$; las variables numéricas mutan dentro de registros de la CPU o en el marco de activación de la pila. Sin embargo, al auditar el proceso desde el sistema operativo, Windows asigna 3.04 MB a Zig y 4.31 MB a Rust. Este diferencial externo representa el Working Set mínimo obligatorio para cargar la imagen binaria en memoria física e inicializar las librerías del sistema. Rust resulta ligeramente más pesado debido a la inclusión nativa de metadatos de seguridad y prevención de pánicos.

2. La Arquitectura de Memoria en Entornos Gestionados: JavaScript vs. Python

El análisis de los lenguajes interpretados desvela el enorme impacto del entorno de ejecución sobre los recursos del sistema, posicionando a JavaScript (Node.js) como un entorno drásticamente más pesado que Python (CPython) en ambas capas de medición, aunque con resultados diametralmente opuestos en rendimiento:

- **Capa Interna (Heap):** Python reporta un uso perfecto de 0.00 MB en su Heap interno. La asignación interna nula se debe a su eficiente motor basado en Conteo de Referencias (Reference Counting); al actualizar la variable de cálculo intermedio, el intérprete destruye inmediatamente el objeto numérico anterior en memoria en un lapso de microsegundos, manteniendo un perfil dinámico plano. JavaScript, en cambio, declara y retiene 4.70 MB en su Heap interno para gestionar las estructuras de datos nativas y optimizaciones dinámicas del motor V8.
- **Capa Externa (OS Real):** Node.js exige un pico masivo de 39.05 MB de RAM física a Windows, mientras que el proceso estático del intérprete de Python se mantiene en un perfil mucho más discreto de 11.89 MB. Esta diferencia abismal obedece a la naturaleza del compilador Just-In-Time (JIT) de JavaScript. Al detectar que el bucle

se ejecuta millones de veces, el optimizador de V8 consume agresivamente la memoria del sistema operativo para almacenar el código máquina compilado al vuelo.

3. Conclusión Arquitectónica

Los datos empíricos demuestran que los entornos gestionados introducen un sobre costo de recursos vinculado a la supervivencia del propio entorno de ejecución. Para cargas de procesamiento matemático masivo, el compilador JIT de JavaScript intercambia un altísimo consumo de RAM (multiplicando su peso por ocho frente al Heap interno) por una gran velocidad (~16.8 segundos). Por su parte, la abstracción pura de Python conserva mejor la memoria global pero, al evaluar cada línea secuencialmente sin compilación al vuelo, se traduce en una penalización de rendimiento de casi 1,100 veces en tiempo en comparación con soluciones compiladas nativamente como Zig y Rust.

Actividad III: Diseño de un DSL

Definición del Entorno Físico: Sistema ECO-GRID

El sistema ECO-GRID constituye una arquitectura avanzada de Gestión de Energía para Microredes Inteligentes (Smart Microgrids), diseñada para supervisar, optimizar y proteger el flujo de energía en entornos industriales con alta demanda operativa. El propósito fundamental de esta infraestructura es garantizar la continuidad del servicio mediante la gestión eficiente de fuentes renovables y sistemas de almacenamiento, minimizando el estrés térmico y los riesgos eléctricos.

Componentes de Hardware Controlados:

El software ECO-GRID actúa como el "cerebro" lógico que interactúa con la siguiente infraestructura física a través de las primitivas de nuestro lenguaje:

- **Paneles Solares (Captación):** Fuentes primarias de generación fotovoltaica que inyectan energía al sistema. El sistema monitorea el flujo de generación en tiempo real.
- **Baterías de Litio (Almacenamiento):** Sistema de almacenamiento de energía (ESS) crítico para la estabilidad de la red. Requieren una monitorización estricta de sus estados de carga para maximizar su vida útil.
- **Sensores Térmicos:** Dispositivos de seguridad instalados en los contenedores de baterías. Su función es reportar variaciones de temperatura que podrían derivar en fallos catastróficos o fugas térmicas.

- **Inversores (Conversión):** Equipos responsables de la conversión de corriente continua a alterna y el acoplamiento de la energía generada con la red industrial o externa.
- **Relés (Actuadores):** Elementos electromecánicos de alta potencia que permiten la conmutación de líneas. Son el músculo físico del sistema, permitiendo aislar sectores de consumo o redirigir el flujo eléctrico bajo demanda o emergencia.

¿Por qué ECO-GRID necesita un lenguaje DSL?

Dado que este entorno físico opera bajo condiciones de estrés crítico, la gestión manual es propensa a errores humanos. Nuestro Lenguaje L (ECO-GRID) fue diseñado específicamente para que cualquier operador de planta pueda configurar las reglas de seguridad, automatizar la refrigeración y gestionar los cortes de carga de forma rápida, legible y, sobre todo, segura.

Especificación Léxica y Morfológica (Manual de Referencia)

El Lenguaje L (ECO-GRID) está diseñado bajo una filosofía de legibilidad extrema para operadores de plantas eléctricas. Prescinde de delimitadores estrictos de cierre de línea y adopta un vocabulario nativo en español para mitigar errores de sintaxis en entornos de estrés operativo. A continuación, se detalla el comportamiento del analizador léxico conceptual y el diccionario de primitivas de hardware del lenguaje.

Alfabeto Aceptado:

Para la escritura del código fuente y nombramiento de identificadores, el lenguaje reconoce los caracteres del alfabeto latino en minúsculas y mayúsculas (**a-z**, **A-Z**), dígitos numéricos (**0-9**) y el carácter de subrayado (**_**). Adicionalmente, el analizador léxico admite

espacios en blanco, comillas dobles (") y signos de puntuación de la codificación estándar (ASCII/UTF-8) **exclusivamente** para la encapsulación y formación de cadenas de texto.

Listado Exhaustivo de Tokens Válidos

El escáner del código fuente clasifica todas las cadenas de texto del programa en las siguientes categorías de tokens (Token Classes):

TOKEN **LIT_NUM** (Literales Numéricos)

Representa valores constantes numéricos. Acepta números enteros para cálculos de capacidad y números de punto flotante para mediciones térmicas de precisión.

- **Ejemplo de uso:**

```
limiteTemperatura = 55.5  
capacidadMinima = 20
```

TOKEN **LIT_BOOL** (Literales Booleanos)

Representan estados lógicos discretos de los componentes. Se prescinde de los tradicionales **true/false** adoptando palabras clave en español adaptadas al control de actuadores.

- **Valores aceptados:** **encendido**, **apagado**.

- **Ejemplo de uso:**

```
estadoRele = encendido
```

TOKEN **LIT_CADENA** (Literales de Texto)

Representan secuencias estáticas de caracteres (strings). El analizador léxico reconocerá como texto plano cualquier cadena que se encuentre encapsulada estrictamente entre comillas dobles (").

- **Ejemplo de uso:**

```
mensajeError = "Fuga térmica detectada en contenedor"
```

TOKEN **LIT_NIVEL** (Literales de Criticidad)

Representan los estados fijos de severidad y urgencia operativa configurados en el sistema de mensajería de la planta. El escáner los reconocerá como palabras reservadas de valor constante.

- **Valores aceptados:** **info**, **advertencia**, **critico**.

- **Ejemplo de uso:**

```
nivelUrgencia = critico
```

TOKEN **ID** (Identificadores)

Nombres definidos por el usuario para variables, colecciones, sensores o sectores específicos de la microred. Para diferenciarlos visualmente de las funciones nativas

del sistema, deben iniciar obligatoriamente con una letra minúscula y utilizar estrictamente la convención de escritura **camelCase**.

- **Ejemplo de uso:**

```
bateriaNorte1 = 100  
sectorIndustrial = apagado
```

TOKEN **ESTRUCTURA** (Palabras de Control de Flujo)

Palabras reservadas por el lenguaje para definir la arquitectura lógica del programa (condicionales, iteraciones y manejo de errores). Deben escribirse obligatoriamente en MAYÚSCULAS para resaltar visualmente la jerarquía del código frente a las variables del operador.

- **Valores aceptados:** SI, ENTONCES, SINO, FIN_SI, MIENTRAS, EJECUTAR, FIN_MIENTRAS, PARA_CADA, EN, FIN_PARA, INTENTAR, EN_CASO_DE_FALLA, FIN_INTENTAR.
- **Comando de interrupción (ROMPER):** Palabra reservada que fuerza la salida inmediata y absoluta del hilo de ejecución de cualquier bucle iterativo, omitiendo las condiciones restantes.

TOKEN **OP** (Operadores de Expresión)

Símbolos y palabras reservadas reconocidos por el analizador léxico para ejecutar operaciones aritméticas, evaluaciones de relación, álgebra booleana y manipulación de cadenas de texto.

- **Asignación:** `=` (Operador único para almacenar valores en identificadores).
- **Aritméticos:** `+` (Suma), `-` (Resta), `*` (Multiplicación), `/` (División) y `%` (Módulo).
- **Concatenación de Cadenas:** El operador `+` presenta sobrecarga para unir texto con identificadores o resultados numéricos. **Regla de precedencia:** Las operaciones matemáticas dentro de una concatenación deben ir entre paréntesis `()`.
- **Relacionales:** `>` (Mayor), `<` (Menor), `>=` (Mayor o igual), `<=` (Menor o igual), `==` (Igualdad estricta), `!=` (Diferencia).
- **Lógicos:** `Y`, `O`, `NO`.
- **Ejemplo de uso:**

```
SI (tempActual > limiteSeguro) ENTONCES
    emitir_alerta(critico, "Límite superado por: " + (tempActual -
limiteSeguro) + " grados.")
```

TOKEN **DELIM** (Delimitadores y Símbolos Especiales)

Definen la separación de sentencias e instrucciones, así como la encapsulación de argumentos y colecciones de hardware.

- **Salto de línea (tecla Enter):** Actúa como el único delimitador válido para indicar el final de una instrucción (carácter interno `\n`).

- **, (Coma):** Actúa como separador secuencial. Se utiliza para dividir múltiples argumentos dentro de una función o separar los elementos individuales al declarar un vector.
- **[y]:** Delimitadores para declarar o indexar vectores (agrupaciones de hardware).
- **(y):** Delimitadores con triple función. Sirven para agrupar expresiones lógicas, pasar parámetros a funciones nativas, y aislar operaciones matemáticas para forzar su precedencia de ejecución (vital al utilizar concatenación de cadenas).
- **Ejemplo de uso:**

```
bateriasCriticas = [bateria1, bateria2, bateria3]
emitir_alerta(critico, "Falla en sector")
```

TOKEN COMENTARIO

Cualquier línea o fragmento de texto precedido por el símbolo de numeral **#**. El analizador léxico descarta automáticamente el resto de los caracteres de esa línea durante la fase de escaneo.

- **Ejemplo de uso:**

```
# Desconectar la carga si ocurre una sobretensión
```

Diccionario de Primitivas de Hardware (Palabras Clave)

El lenguaje incorpora un conjunto cerrado de funciones nativas (**TOKEN PRIMITIVA**) que interactúan directamente con los drivers lógicos y actuadores mecánicos de la planta ECO-GRID. Para distinguirse del código del usuario (**camelCase**), el motor del sistema preserva estas invocaciones nativas en convención **snake_case**.

init_grid()

- **Descripción:** Inicializa, verifica y calibra todos los controladores lógicos y buses de comunicación de la microrred. Debe ser de manera obligatoria la primera instrucción ejecutable de cualquier programa.
- **Retorno:** Ninguno.
- **Pseudo-notación:** **init_grid()** <\n>.
- **Ejemplo válido:**
- Fragmento de código

```
init_grid()
```

leer_temperatura(bateriaId)

- **Descripción:** Consulta el sensor térmico integrado en una celda o contenedor específico de almacenamiento.
- **Retorno:** Flotante (Valor térmico preciso en grados Celsius °C).
- **Pseudo-notación:** **leer_temperatura(<TOKEN ID_bateria>)**.
- **Ejemplo válido:**

```
tempActual = leer_temperatura(bancoLitioSur)
```

estado_carga(bateriaId)

- **Descripción:** Retorna el nivel relativo de la energía remanente dentro de un banco de celdas específico.
- **Retorno:** Entero (Valor porcentual acotado en el rango de 0 a 100).
- **Pseudo-notación:** `estado_carga( <TOKEN ID_bateria> )`.
- **Ejemplo válido:**
- Fragmento de código

```
cargaDisponible = estado_carga(bancoLitioSur)
```

flujo_actual(sensorId)

- **Descripción:** Realiza una lectura instantánea del paso de potencia eléctrica a través de los sensores e inversores de acoplamiento.
- **Retorno:** Entero (Potencia medida en kilovatios [kW]. Los valores positivos representan inyección/generación de energía y negativos consumo).
- **Pseudo-notación:** `flujo_actual( <TOKEN ID_sensor> )`.
- **Ejemplo válido:**

```
generacionSolar = flujo_actual(sensorPaneles1)
```

conmutar_linea(sectorId, estado)

- **Descripción:** Modifica el estado físico de los relés electromecánicos de alta potencia para conectar o aislar un sector de consumo de la red.
- **Retorno:** Ninguno.
- **Pseudo-notación:** `conmutar_linea( <TOKEN ID_sector> , <TOKEN LIT_BOOL> ) <\n>`.
- **Ejemplo válido:**

```
# Aislar el área de oficinas de la red principal
conmutar_linea(sectorOficinas, apagado)
```

activar_refrigeracion(bateriaId)

- **Descripción:** Envía una señal de activación forzada a los extractores y sistemas de enfriamiento auxiliar del contenedor especificado.
- **Retorno:** Ninguno.
- **Pseudo-notación:** `activar_refrigeracion( <TOKEN ID_bateria> ) <\n>`.
- **Ejemplo válido:**

```
activar_refrigeracion(bancoLitioSur)
```

emitir_alerta(nivelCriticidad, mensaje)

- **Descripción:** Envía de forma síncrona una notificación estructurada para ser desplegada inmediatamente en la pantalla de alarmas de la HMI. Permite la concatenación de texto estático con identificadores dinámicos o con el resultado de operaciones aritméticas usando el operador **+**.
- **Retorno:** Ninguno.

A. Concatenación directa con variable simple Se utiliza para unir el texto con el valor de un identificador sin ejecutar cálculos matemáticos.

- **Pseudo-notación:** `emitir_alerta( <TOKEN LIT_NIVEL> , <TOKEN LIT_CADENA> + <TOKEN ID> ) <\n>`.
- **Ejemplo válido:**

```
# Envía el texto junto al nombre específico de la batería
emitir_alerta(critico, "Fuga térmica detectada en: " + bancoLitioSur)
```

B. Concatenación con operation matemática Requiere el uso obligatorio de paréntesis para aislar la expresión aritmética, forzando al motor a resolver el cálculo antes de unirlo al texto.

- **Pseudo-notación:** `emitir_alerta( <TOKEN LIT_NIVEL> , <TOKEN LIT_CADENA> + ( <operacion_aritmetica> ) ) <\n>`.
- **Ejemplo válido:**

```
# El compilador resuelve primero la resta y luego une el resultado numérico
al texto

emitir_alerta(advertencia, "Límite térmico superado por: " + (tempActual -
55.0))
```

hora_actual()

- **Descripción:** Consulta el reloj interno en tiempo real de los servidores de la planta para la toma de decisiones cronológicas.
- **Retorno:** Entero (Valor numérico de 0 a 23, representando la hora militar local).
- **Pseudo-notación:** `hora_actual()`.
- **Ejemplo válido:**

```
SI (hora_actual() >= 18 Y hora_actual() <= 23) ENTONCES
    conmutar_linea(sectorIndustrial, apagado)
FIN_SI
```

esperar(segundos)

- **Descripción:** Pausa la ejecución del hilo de monitorización. Obligatoria en bucles continuos para evitar el colapso del CPU.
- **Retorno:** Ninguno.
- **Pseudo-notación:** `esperar( <TOKEN LIT_NUM_entero> ) <\n>`.
- **Ejemplo válido:**

```
esperar(60) # Pausa el sistema durante un minuto
```

Gramática Sintáctica Abstracta

Una vez que el analizador léxico ha tokenizado el código fuente, el analizador sintáctico de ECO-GRID evalúa el orden lógico de las instrucciones para construir el Árbol de Sintaxis Abstracta (AST).

Para garantizar que la máquina interprete los comandos sin ambigüedades, se establece el siguiente orden lógico obligatorio, descrito mediante pseudo-notación estructural y lenguaje natural.

Estructura de Declaración y Asignación

Define la regla gramatical para almacenar valores numéricos (**LIT_NUM**), booleanos (**LIT_BOOL**), cadenas de texto (**LIT_CADENA**), agrupaciones de hardware (**vector**) o lecturas de sensores en la memoria del sistema.

- **Orden Lógico Estricto:** La sentencia debe iniciar obligatoriamente con un **TOKEN ID** (en convención **camelCase**), seguido del operador de asignación **=**, seguido de una expresión válida, y finalizar con el delimitador de salto de línea **\n**.
- **Pseudo-notación:** **<identificador_camelCase> = <literal | primitiva | operacion | vector> <\n>**
- **Ejemplo válido:**

```
temperaturaMaxima = 55.0
mensajePeligro = "Temperatura crítica alcanzada"
cargaActual = estado_carga(bateriaPrincipal)
bateriasNorte = [batNorte1, batNorte2]
```


Estructuras Condicionales (Toma de Decisiones)

Regula cómo el sistema evalúa el entorno para ejecutar acciones de respuesta (como apagar un relé si hay sobrecalentamiento).

- **Orden Lógico Estricto:** Inicia con **SI**, seguido de una condición lógica entre paréntesis (), la palabra **ENTONCES** y un salto de línea. El bloque **SINO** es opcional para manejar condiciones contrarias. Todo el bloque se cierra con **FIN_SI**.
- **Pseudo-notación:**

```
SI ( <expresion_logica> ) ENTONCES <\n>  
    <bloque_instrucciones_verdadero> <\n>  
SINO <\n>  
    <bloque_instrucciones_falso> <\n>  
FIN_SI <\n>
```

- **Ejemplo válido:**

```
SI (temperaturaActual > 55.0) ENTONCES  
    conmutar_linea(sectorIndustrial, apagado)  
SINO  
    conmutar_linea(sectorIndustrial, encendido)  
FIN_SI
```

Estructuras Iterativas (Bucles)

Garantizan la monitorización continua y autónoma del hardware. El lenguaje define dos estructuras jerárquicas sin ambigüedad para ciclos iterativos.

A. Bucle Condicional Continuo (MIENTRAS) Se ejecuta indefinidamente mientras la condición evaluada retorne un valor lógico verdadero.

- **Pseudo-notación:**

```
SI ( <expresion_logica> ) ENTONCES <\n>
    <bloque_instrucciones_verdadero> <\n>
SINO <\n>
    <bloque_instrucciones_falso> <\n>
FIN_SI <\n>
```

- **Ejemplo válido (Demostrando control de tiempo, concatenación y ruptura de emergencia) :**

```
SI (temperaturaActual > 55.0) ENTONCES
    conmutar_linea(sectorIndustrial, apagado)
SINO
    conmutar_linea(sectorIndustrial, encendido)
FIN_SI
```

B. Bucle de Colección (PARA_CADA) Estructura específica diseñada para recorrer agrupaciones masivas de hardware (vectores), extrayendo un elemento a la vez para su evaluación individual.

- **Pseudo-notación:**

```
SI ( <expresion_logica> ) ENTONCES <\n>
    <bloque_instrucciones_verdadero> <\n>
SINO <\n>
    <bloque_instrucciones_falso> <\n>
FIN_SI <\n>
```

- **Ejemplo válido:**

```
SI (temperaturaActual > 55.0) ENTONCES
    conmutar_linea(sectorIndustrial, apagado)
SINO
    conmutar_linea(sectorIndustrial, encendido)
FIN_SI
```

Estructura Sintáctica de Resiliencia (Manejo de Errores)

Para evitar la detención total del software ante la pérdida de conexión con un sensor físico, la gramática incorpora una estructura de control de excepciones.

- **Orden Lógico Estricto:** El bloque primario se inicia con **INTENTAR**. Si alguna instrucción falla o devuelve un error de hardware, el flujo salta inmediatamente a la estructura **EN_CASO_DE_FALLA**, ejecutando el bloque de contingencia. Se cierra con **FIN_INTENTAR**.
- **Pseudo-notación:**

```
INTENTAR <\n>
    <bloque_de_instrucciones_primarias> <\n>
EN_CASO_DE_FALLA <\n>
    <bloque_de_contingencia_y_alertas> <\n>
FIN_INTENTAR <\n>
```

- **Ejemplo válido:**

```
INTENTAR
    temp = leer_temperatura(bateriaNorte)
EN_CASO_DE_FALLA
    emitir_alerta(critico, "Fallo de comunicación con el sensor en: " +
bateriaNorte)
FIN_INTENTAR
```

Resolución de Escenarios Operativos Críticos

Escenario Operativo A: Prevención de Fuga Térmica

Objetivo: Monitorización iterativa para prevenir el sobrecalentamiento de las baterías.

Lógica Implementada: El programa inicia con la calibración general del sistema a través de `init_grid()`. Se establece un bucle de monitoreo continuo **MIENTRAS** que evalúa constantemente una celda crítica (`bateriaPrincipal`). La telemetría se encapsula en una estructura **INTENTAR** para prever fallas de comunicación con el sensor. Si la temperatura supera el umbral estricto de 55.0°C , el script ejecuta de forma inmediata el protocolo de alivio de carga:

1. Activa mecánicamente los extractores con `activar_refrigeracion()`.
2. Desconecta la captación de energía fotovoltaica (`sensorPaneles1`) cambiando su estado a `apagado` para frenar el ingreso de calor por carga.
3. Desvía el consumo aislando el `sectorIndustrial` (`apagado`) y conectando la `redComercial` de respaldo (`encendido`).

Posteriormente, el sistema ejecuta una pausa prudencial de 10 segundos con `esperar()`. Si tras este periodo la temperatura se mantiene por encima de los 55.0°C , el programa utiliza la instrucción de interrupción forzada **ROMPER** para abortar el ciclo ordinario e ingresar inmediatamente a un estado de bloqueo absoluto, emitiendo una alerta recurrente de nivel `critico` detallando el exceso térmico exacto.

Código Fuente:

```

init_grid()

monitoreoTermico = encendido

MIENTRAS (monitoreoTermico == encendido) EJECUTAR
    INTENTAR
        tempActual = leer_temperatura(bateriaPrincipal)

        SI (tempActual > 55.0) ENTONCES
            # 1. Activar refrigeracion y aislar fuentes de calor
            activar_refrigeracion(bateriaPrincipal)
            conmutar_linea(sensorPaneles1, apagado)

            # 2. Desviar el consumo industrial a la red comercial de respaldo
            conmutar_linea(sectorIndustrial, apagado)
            conmutar_linea(redComercial, encendido)

            # 3. Periodo de verificacion de persistencia del peligro

            esperar(10)
            tempVerificacion = leer_temperatura(bateriaPrincipal)
            SI (tempVerificacion > 55.0) ENTONCES
                emitir_alerta(critico, "Peligro persistente de fuga. Abortando
ciclo. Exceso: " + (tempVerificacion - 55.0))
                ROMPER
            FIN_SI
        FIN_SI
    EN_CASO_DE_FALLA
        emitir_alerta(critico, "Fallo de lectura en sensor termico de: " +
bateriaPrincipal)
    FIN_INTENTAR

    esperar(60)
FIN_MIENTRAS

```

Escenario Operativo B: Balance de Carga y Optimización Energética

Objetivo: Toma de decisiones autónoma para inyectar excedentes o aislar sectores críticos.

Lógica Implementada: El script recopila las variables de entorno actuales (hora militar y métricas de potencia de los componentes). Todo el flujo de evaluación se protege con un bloque **INTENTAR**. El sistema decide autónomamente entre dos estados lógicos excluyentes:

- **Condición de Superávit:** Si el almacenamiento de la **bateriaPrincipal** es óptimo (superior al **90%**) **Y** la generación registrada en los paneles (**flujoSolar**) es mayor al consumo demandado por la planta (**demandaInterna**), los relés automáticos se accionan conectando la **redPublica** a estado **encendido** para vender el excedente energético. Se emite un reporte tipo **info**.
- **Condición de Contingencia Nocturna:** Si el almacenamiento desciende de un límite crítico (menor al **20%**) **Y** el reloj de la planta se encuentra en horas de alta demanda nocturna (entre las 18 y las 23 horas), el sistema prioriza la seguridad operativa. Ejecuta la desconexión inmediata de los sectores prescindibles (**sectorIndustrial** y **sectorOficinas**) a estado **apagado**. Esto aísla las cargas pesadas, garantizando el suministro remanente de manera exclusiva para las áreas designadas como intocables en el manual de contingencias de la fábrica (**sectorMedico** y **sectorServidores**).

Código Fuente:

```

init_grid()

horaMilitar = hora_actual()

INTENTAR

    cargaActual = estado_carga(bateriaPrincipal)

    flujoSolar = flujo_actual(sensorPaneles1)

    demandaInterna = flujo_actual(sensorConsumoInterno)

    # CASO 1: Inyeccion/Venta de excedentes por superavit energetico
    SI (cargaActual > 90 Y flujoSolar > demandaInterna) ENTONCES
        conmutar_linea(redPublica, encendido)

        emitir_alerta(info, "Inyectando excedente a red publica. Potencia: " +
(flujoSolar - demandaInterna))

    FIN_SI

    # CASO 2: Aislamiento preventivo por baja reserva en horas pico nocturnas
    SI (cargaActual < 20 Y horaMilitar >= 18 Y horaMilitar <= 23) ENTONCES
        # Desconectar cargas pesadas no esenciales

        conmutar_linea(sectorIndustrial, apagado)

        # Asegurar flujo electrico a las areas criticas de la organizacion

        conmutar_linea(sectorMedico, encendido)

        conmutar_linea(sectorServidores, encendido)

        emitir_alerta(advertencia, "Racionamiento activo. Suministro preservado en
areas criticas. Bateria: " + cargaActual)

    FIN_SI
EN_CASO_DE_FALLA

    emitir_alerta(critico, "Error de comunicacion en bus de datos de optimizacion.
Protocolo bloqueado.")

FIN_INTENTAR

```

Conclusiones

La presente investigación permite concluir que los lenguajes de programación no operan como simples traductores de lógica algorítmica, sino como complejas infraestructuras cuyas teorías gramaticales y morfológicas definen los límites físicos del software.

Al vincular la teoría gramatical con las observaciones prácticas de rendimiento compilado, el benchmarking sobre la Conjetura de Collatz evidenció cómo la construcción del Árbol de Sintaxis Abstracta (AST) impacta directamente en los recursos del sistema operativo. Los lenguajes de compilación nativa (Zig y Rust) demostraron que una gramática estricta orientada a la seguridad de memoria permite resolver el procesamiento intensivo en tiempo récord (~1.4 segundos) con un consumo interno nulo. En contraste, los entornos gestionados desvelaron el alto costo de sus arquitecturas: la optimización Just-In-Time de JavaScript intercambia un consumo masivo de RAM física (39.05 MB) por velocidad, mientras que la interpretación pura de Python sacrifica drásticamente el tiempo de ejecución al prescindir de compilación al vuelo.

Finalmente, el diseño de nuestro Lenguaje L (ECO-GRID) materializa la importancia de la abstracción sintáctica en el diseño de lenguajes orientados a dominios específicos (DSL). Al omitir delimitadores complejos y emplear palabras de control nativas en español, se demostró que manipular la gramática abstracta es vital para dotar a los operadores industriales de una interfaz resiliente. Esta abstracción sintáctica previene errores humanos al automatizar protocolos críticos como la prevención de fugas térmicas, confirmando que el diseño de un lenguaje dictamina de forma directa la seguridad y eficiencia de los sistemas en el mundo real.

Referencias Bibliográficas

Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2.^a ed.). Pearson Educación. Recuperado de <https://es.scribd.com/doc/237436044/Compiladores-Principios-Tecnicas-Y-Herramientas>

García, F., y otros. (2012). Desarrollo de un lenguaje de dominio específico para sistemas de gestión. *Revista Iberoamericana de Tecnologías del Aprendizaje*. Recuperado de <https://dialnet.unirioja.es/descarga/articulo/3869349.pdf>

Juiz, C., Molero, X., y Rodeño, M. J. (2004). *Evaluación y modelado del rendimiento de los sistemas informáticos*. Prentice Hall. Recuperado de https://www.researchgate.net/profile/Carlos_Juiz/publication/266565524_Evaluacion_y_Modelado_del_Rendimiento_de_los_Sistemas_Informaticos/links/5ad3284baca272fdaf79fbd9/Evaluacion-y-Modelado-del-Rendimiento-de-los-Sistemas-Informaticos.pdf

Mozilla Developer Network [MDN]. (2024). *Referencia del lenguaje JavaScript*. MDN Web Docs. Recuperado de <https://developer.mozilla.org/es/docs/Web/JavaScript>

Python Software Foundation. (2024). *La biblioteca estándar de Python*. Documentación oficial de Python 3. Recuperado de <https://docs.python.org/es/3/library/index.html>

Rust Comunidad Hispana. (2024). *El Lenguaje de Programación Rust*. Documentación oficial traducida al español. Recuperado de <https://rust-lang-es.org/rust-book-es/>

Trejos Buriticá, O. I. (2013). Consideraciones sobre la evolución del pensamiento a partir de los paradigmas de programación de computadores. *Con-ciencias*, 68-75. Recuperado de <https://dialnet.unirioja.es/descarga/articulo/3957052.pdf>

Trejos Buriticá, O. I. (2013). Relaciones de aprendizaje significativo entre dos paradigmas de programación a partir de dos lenguajes de programación. *Revista de Investigaciones de la Universidad Católica de Manizales*, 93-100. Recuperado de <https://dialnet.unirioja.es/descarga/articulo/4778514.pdf>

Zig Software Foundation. (2024). *Documentación y referencia del lenguaje Zig*. Recuperado de <https://ziglang.org/documentation/0.16.0/>

Trejos Buriticá, O. I. (2011). Consideraciones sobre la evolución del pensamiento humano a partir de los paradigmas de programación. *Scientia et Technica*, 16(47), 164-169. Recuperado de <https://dialnet.unirioja.es/descarga/articulo/4725669.pdf>

Torres, H. L., y Silva, A. M. (2014). Eficiencia algorítmica en aplicaciones de grafos. *Ingeniería Solidaria*, 10(17), 75-81. Recuperado de <https://www.redalyc.org/pdf/4139/413937007009.pdf>

Escartí, R., y otros. (2012). Desarrollo de un lenguaje de dominio específico para sistemas de gestión de aprendizaje y su herramienta de implementación. *Revista Iberoamericana de Tecnologías del Aprendizaje*. Recuperado de <https://dialnet.unirioja.es/descarga/articulo/3869349.pdf>